SOFTWARE REQUIREMENTS SPECIFICATION

Tour Management System (Python)

Document ID:	SRS-TMS-2026
Prepared By:	Giridharan
Target Environment:	Python Runtime Environment (Desktop/Web)
Date of Creation:	May 18, 2026
Document Version:	1.1 (Updated Name PDF)

1. Introduction

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) is to present a clean, comprehensive, and well-aligned blueprint for the development of the Tour Management System. This Python-based application serves as a unified digital platform to automate and optimize the foundational operations of travel agencies, enabling users to search, book, and manage dynamic tour packages seamlessly.

1.2 Scope

The scope of the Tour Management System captures both customer-facing interfaces and administrative back-office execution systems. Customers are empowered with front-end catalogs to seamlessly view available packages, customize profile configurations, track historical invoices, and place secure seat reservations. Concurrently, system Administrators are provided explicit back-end access modules to provision new packages, modify active prices, monitor customer registration metrics, and print automated internal ledger logs.

1.3 Objectives

- **Provide Easy Access to Tour Information:** Enable quick, organized browsing of structured travel itineraries.
- **Enable Online Booking:** Replace physical, paper-bound reservation models with dynamic live booking steps.
- **Reduce Manual Work:** Automate mathematical invoicing computations, tax overhead calculations, and seat counters.
- **Store Customer & Booking Data Securely:** Safeguard private access vectors and maintain transactional records cleanly.
- **Generate Operational Reports:** Empower managerial oversight with structured overview logs capturing users, tours, and revenue tracks.

2. Overall Description

2.1 Product Perspective

The Tour Management System functions as an independent, self-contained relational application built entirely using Python. Business and operational rules are evaluated within a specialized logic engine that bridges user UI forms to an underlying database architecture. The system stores real-time updates regarding customer

details, login sessions, tour inventories, and transactional statuses to guarantee immediate updates across all views.

2.2 User Types

The system defines two discrete user roles with explicit operational matrices:

> **1. Administrator (Admin Profile):**
>
> • Authenticates through high-security backend interfaces.
>
> • Executes full CRUD capabilities (Add, Update, Delete) on tour catalogs.
>
> • Monitors master booking spreadsheets and checks customer profiles.
>
> • Compiles systemic reports evaluating financial revenue pipelines.
>
> **2. Customer (Traveler Profile):**
>
> • Registers personal profile parameters onto the local database.
>
> • Conducts dynamic filter operations matching specific locations and budgets.
>
> • Commits safe tour checkouts and confirms digital passenger metrics.
>
> • Reviews historic personal reservation sheets and ledger data.

3. Functional Requirements

3.1 Registration Module

The application must provide a dedicated signup panel to capture and validate new traveler criteria. Before record storage, fields must be mapped to the following attributes: Name, Email Identity, Contact Phone Number, and Private Password.

3.2 Login Module

Access control parameters must evaluate provided usernames (or email tokens) against hashed values in the relational database, generating active application sessions upon verification.

3.3 Tour Package Module

Administrators must have input panels to create and manipulate catalog offerings. Every tour card record must keep explicit properties, including: Target Destination, Flat Package Price, Trip Duration (Days/Nights), and Comprehensive Description.

3.4 Search Module

The customer terminal must support multi-attribute querying pipelines to filter active tours based on: Target Destination matching, Preferred Calendar Date windows, and Specified Maximum Financial Budget limits.

3.5 Booking Module

Allows authorized customers to select specific packages, execute seat-count calculations, verify reservation confirmations, and open localized digital summaries of pending itineraries.

3.6 Payment Module

Implements logic loops to register mock financial transactions, evaluate payment approvals, generate transaction IDs, and switch database row fields to 'Paid' or 'Pending' status values.

3.7 Report Module

Provides executive command lines for administrators to run aggregation scripts, producing formatted analytical summaries of Registered System Users, Volumetric Booking Logs, and Combined Financial Revenue gains.

4. Non-Functional Requirements

4.1 Performance

The response latency across database queries, view loading states, and calculation refreshes must execute in less than 3.0 seconds under peak thread loading metrics.

4.2 Security

All passwords must never be stored as clear strings. The Python core layer must execute one-way cryptographic hashing algorithms (e.g., hashlib SHA-256 or bcrypt) prior to persistence.

4.3 Reliability & Usability

The application must operate continuously without interface crashes, displaying clean error handlers during network drops. The visual interface must follow intuitive layouts, enabling untrained users to complete bookings within minutes.

5. Technology Requirements

The table below details the systemic, aligned architectural specifications selected for this implementation:

System Component Layer	Selected Technology Requirement	Implementation Role
Core Programming Language	Python 3.10+	Handles all system processes, date processing, and calculations.
Frontend UI Framework	Tkinter / HTML, CSS	Renders the desktop interface forms or web views cleanly.
Backend System Logic	Python Native Modules	Evaluates validation steps and routes operational requests.
Database Storage Engine	MySQL or SQLite	Maintains relational database structures and constraints.
Integrated Dev Env (IDE)	VS Code / PyCharm	Used as the primary environment for coding and unit testing.

6. Hardware Requirements

To run the Python application engine without computational bottlenecks, the environment must meet or exceed the following specifications:

Hardware Component	Minimum Required Specification	Recommended Target Specification
Processor (CPU)	Intel i3 Core / AMD Equivalent	Intel i5 Core / Apple M-Series or superior
System Memory (RAM)	4 GB Minimum Space Allocation	8 GB RAM or above for multi-threaded testing
Storage Space (Hard Disk)	500 GB Available Capacity Drive	256 GB / 512 GB Solid State Drive (SSD)

7. Conclusion

The Python-based Tour Management System systematically streamlines tour planning and package oversight. By structuring complex data matrices into modular user paths, the platform minimizes human operational errors, improves database storage security, and upgrades traveler convenience. This SRS document provides an exact architectural foundation, ensuring clear alignment across all upcoming coding, database generation, and testing cycles.